คู่มือเตรียมสอบโปรเจกต์: ระบบจัดการร้านเกมคาเฟ่ LETSPLAY

แนวคำถาม คำตอบเชิงลึก และยุทธวิธีการนำเสนอสำหรับวิชา Web Application & Database Systems

สแตกเทคโนโลยี: PHP 8.4, Laravel 13, SQLite, Blade, CSS

สถาปัตยกรรม: Pure Laravel MVC + Service Layer Pattern

ฐานข้อมูล: SQLite (11 ตารางหลัก, Audit Logs, Price Snapshots)

ชุดทดสอบ: Pest Automated Test Suite (53 ผ่านครบ 100%)

1. การแบ่งบทบาทหน้าที่ของสมาชิกในทีม (5 คน)

สมาชิก	ส่วนที่รับผิดชอบ	ไฟล์โค้ดหลัก	ตารางฐานข้อมูลและประเด็นเด่น
1. ปาล์ม	Web: Customer Experience, Dashboard, Seat Map, Auth	`DashboardController.php` `SeatController.php` `dashboard.blade.php`	ตาราง `users`, `seat_sessions` ระบบ Check-in, Check-out, Auto-cut session เมื่อเงินหมด
2. ตอง	Web: Food POS, Cart, Wallet & Packages	`FoodOrderController.php` `WalletController.php` `food-order.blade.php`	ตาราง `orders`, `packages`, `user_packages` กลไก Available Balance กันเงินค่าเครื่องก่อนสั่งอาหาร
3. ดิว	Web: Staff Monitor, Kitchen Queue, Admin Report	`StaffController.php` `AdminController.php` `kitchen-queue.blade.php`	ตาราง `zones`, `seats`, `products` การยืนยันรับเงินสด, คืนสต็อกเมื่อ Cancel, สรุปยอดขาย
4. บอล	Database: Master Data & Sessions	`create_gaming_cafe_tables.php` `Zone.php`, `Seat.php`, `Package.php`	ตาราง `users`, `zones`, `seats`, `packages`, `user_packages`, `seat_sessions` ความสัมพันธ์ 1-N, Rich Pivot, Foreign Key Constraints
5. นะโม	Database: Transactions, Orders & Products	`create_gaming_cafe_tables.php` `Order.php`, `Product.php`, `WalletTransaction.php`	ตาราง `wallet_transactions`, `categories`, `products`, `orders`, `order_items` Audit Trail, Snapshot Pattern, SQL Aggregate

2. หมวดโครงสร้างฐานข้อมูลและการออกแบบ (Database & Schema Design)

DB คำถามที่ 1: ทำไมตาราง `seat_sessions` ต้องมีคอลัมน์ `rate_snapshot` ทั้งที่เครื่องผูกกับโซนอยู่แล้ว?

คำตอบ: เพื่อแก้ปัญหา Price Change Propagation หากเราดึงราคาโดยการ JOIN ไปยังตาราง `zones` ตลอดเวลา เมื่อทางร้านปรับขึ้นราคาค่าบริการของโซนในภายหลัง ประวัติการเล่นเก่าในอดีตหรือเซสชันที่ลูกค้ายังเล่นไม่เสร็จจะถูกคำนวณด้วยราคาใหม่ทันที การบันทึก `rate_snapshot` ไว้ในตาราง `seat_sessions` ณ วินาทีที่กดเปิดเครื่อง ทำให้คิดเงินด้วยราคาที่เป็นธรรมตามสัญญาตอนเช็คอินเสมอ หลักการนี้ถูกใช้กับคอลัมน์ `unit_price` ในตาราง `order_items` เพื่อตรึงราคาสินค้า ณ ตอนสั่งซื้อเช่นกัน

DB คำถามที่ 2: ทำไมต้องแยกตาราง `orders` ออกจาก `order_items` ?

คำตอบ: เป็นไปตามหลัก Normalization ขั้นที่ 1 (1NF) และขั้นที่ 3 (3NF) คำสั่งซื้อ 1 ครั้ง (Order) สามารถประกอบด้วยสินค้าได้หลายรายการ (Many Items) หากเก็บในตารางเดียวจะเกิด Repeating Group หรือต้องเก็บข้อมูลเป็นสตริง ซึ่งไม่สามารถคำนวณผลรวม ค้นหา หรือตัดสต็อกรายตัวผ่าน SQL ได้ การแยก `orders` เป็นส่วนหัวบิล (Header) และ `order_items` เป็นรายการย่อย (Line Items) ช่วยให้บันทึก `quantity` , `unit_price` , และ `subtotal` ได้อย่างถูกต้อง

DB คำถามที่ 3: ทำไม Foreign Key ส่วนใหญ่ใช้ `restrictOnDelete()` แต่ระหว่าง `orders` กับ `order_items` ใช้ `cascadeOnDelete()` ?

คำตอบ: เพื่อรักษาความถูกต้องของข้อมูล (Data Integrity):

- `restrictOnDelete()` : ใช้กับ `users` , `seats` , `products` เพื่อห้ามลบข้อมูลหลักหากยังมีประวัติทางการเงินหรือประวัติการเล่นผูกอยู่ ป้องกันการเกิดข้อมูลกำพร้า (Orphan Records)
- `cascadeOnDelete()` : ใช้กับ `order_items` ที่ผูกกับ `orders` เนื่องจากรายการย่อยในบิลเป็นส่วนประกอบเฉพาะของบิลนั้น หากหัวบิลถูกสั่งลบ รายการย่อยต้องถูกลบตามทันที

DB คำถามที่ 4: ตาราง `user_packages` เป็น Pivot Table ธรรมดาหรือไม่ แตกต่างอย่างไร?

คำตอบ: เป็น Rich Pivot Table ไม่ใช่ Pivot Table แบบคู่ไอดีธรรมดา ตารางนี้มี Primary Key `id` ของตัวเอง และมีแอตทริบิวต์ `remaining_minutes` , `purchased_at` , และ `expired_at` ทำให้ลูกค้ารายเดียวสามารถซื้อแพ็กเกจชนิดเดียวกันซ้ำได้หลายครั้ง โดยระบบจะแยกตัดเวลาและนับวันหมดอายุของแต่ละแพ็กเกจออกจากกันอย่างอิสระ

DB คำถามที่ 5: ทำไมต้องมีตาราง `wallet_transactions` ทั้งที่มีคอลัมน์ `balance` ในตาราง `users` อยู่แล้ว?

คำตอบ: ทำหน้าที่เป็น Audit Trail คอลัมน์ `users.balance` เก็บเพียงยอดเงินปัจจุบัน แต่ตาราง `wallet_transactions` เก็บบันทึกประวัติทุกการเคลื่อนไหวของเงิน ทั้งประเภทรายการ (`topup` , `deduct`), จำนวนเงิน, ที่มาของรายการ (`ref_type` : session, food_order, package_purchase) และเลขอ้างอิง (`ref_id`) ทำให้สามารถตรวจสอบที่มาที่ไปของยอดเงินย้อนหลังได้ตลอดเวลา

3. หมวด Business Logic, Concurrency & ป้องกัน Race Condition

LOGIC **คำถามที่ 1: ถ้าลูกค้าเล่นโหมด Pay-as-you-go มีเงิน 100 บาท เล่นไปแล้ว 60 บาท แล้วไปกดสั่งอาหาร 80 บาท ระบบจะยอมให้สั่งหรือไม่?**

คำตอบ: ระบบจะปฏิเสธคำสั่งซื้อทันที ในไฟล์ `FoodOrderController.php` มีเมธอด `calculateAvailableBalance()`:

```
$usedMinutes = max(1, (int) ceil($startTime→diffInSeconds(Carbon::now()) / 60));
$estimatedCost = round(($usedMinutes / 60) * (float) $session→rate_snapshot, 2);
return max(0.00, round($balance - $estimatedCost, 2));
```

ระบบจะคำนวณค่าเครื่องที่เกิดขึ้นแล้ว (60 บาท) และกันออกจากยอดเงินในกระเป๋า (100 บาท) ทำให้เหลือ Available Balance สั่งอาหารได้เพียง 40 บาท เมื่อลูกค้าสั่ง 80 บาท ระบบจะแจ้งเตือนว่ายอดเงินไม่เพียงพอเนื่องจากต้องกันเงินไว้จ่ายค่าเครื่องคอมพิวเตอร์ ป้องกันเงินติดลบเมื่อเช็คเอาท์

CONCURRENCY **คำถามที่ 2: ถ้าลูกค้า 2 คนเปิดหน้าผังที่นั่งพร้อมกัน เห็นเครื่องเดียวกันว่างอยู่ แล้วกดเช็คอินในเสี้ยววินาทีเดียวกัน ระบบจัดการอย่างไร?**

คำตอบ: ป้องกันด้วย Atomic Database Query ใน `SeatController.php` บรรทัดที่ 123-129:

```
$updated = Seat::where('id', $seat→id)
    →where('status', 'available')
    →update(['status' ⇒ 'occupied']);

if (! $updated) {
    return redirect()→back()→with('error', 'ที่นั่งนี้ถูกผู้อื่นเช็คอินไปก่อนหน้าแล้ว');
}
```

คำสั่ง SQL `UPDATE seats SET status = 'occupied' WHERE id = ? AND status = 'available'` ทำงานระดับ Atomic Request แรกที่ไปถึงฐานข้อมูลจะอัปเดตสำเร็จ (คืนค่า 1) ส่วน Request ที่สองจะไม่พบแถวที่ตรงกับเงื่อนไข (คืนค่า 0) ทำให้ถูกปฏิเสธทันที ป้องกัน Race Condition ได้ 100%

CONCURRENCY **คำถามที่ 3: หากสต็อกสินค้าเหลือ 1 ชิ้น แต่มีลูกค้า 2 คนกดสั่งพร้อมกัน จะเกิดปัญหาสต็อกติดลบหรือไม่?**

คำตอบ: ไม่ติดลบ เพราะใน `FoodOrderController.php` โค้ดถูกครอบด้วย `DB::transaction()` และใช้ `lockForUpdate()` บนตารางสินค้า:

```
$product = Product::where('id', $productId)→lockForUpdate()→first();
if ($product→stock_quantity < $quantity) {
    throw new \Exception('สินค้า ' . $product→name . ' มีไม่พอ');
}
```

ฐานข้อมูลจะล็อกแถวสินค้าไว้ ทำให้ Request ที่สองต้องรอจนกว่า Transaction แรกจะเสร็จสิ้น เมื่อถึงคิว Request ที่สอง สต็อกจะกลายเป็น 0 และระบบจะโยน Exception ยกเลิกการสั่งซื้อและ Rollback ข้อมูลทั้งหมด

LOGIC **คำถามที่ 4: กรณีลูกค้าสั่งอาหารแบบชำระเงินสด พนักงานสามารถเริ่มทำอาหารหรือเสิร์ฟก่อนได้หรือไม่?**

คำตอบ: ไม่ได้ ใน `StaffController.php` เมธอด `updateOrderStatus` มีการตรวจสอบ:

```
if (in_array($status, ['preparing', 'served']) && $order→payment_status ≠= 'paid') {
    return redirect()→back()→with('error', 'ไม่สามารถเปลี่ยนสถานะอาหารได้ เนื่องจากยังไม่ได้ยืนยันรับเงินสด');
}
```

พนักงานต้องกดปุ่มยืนยันการรับเงินสดผ่าน `confirmCashPayment()` ให้ `payment_status = 'paid'` เสียก่อน จึงจะอนุญาตให้เปลี่ยนสถานะเป็นทำอาหารหรือเสิร์ฟได้ ป้องกันปัญหาทำอาหารไปแล้วแต่ลูกค้าไม่จ่ายเงิน

4. หมวดสถาปัตยกรรมซอฟต์แวร์ MVC & Laravel

MVC **คำถามที่ 1: ทำไมโค้ดการปิดเครื่องและคิดเงินถึงถูกแยกออกมาเป็น Service Class `EndSeatSession` ?**

คำตอบ: เพื่อรักษาหลัก DRY (Don't Repeat Yourself) และความสม่ำเสมอของ Business Logic การปิดเครื่องในระบบนี้เกิดขึ้นได้จาก 3 ทาง:

- ลูกค้ากดเช็คเอาท์เองจากหน้า Dashboard (`DashboardController.php`)
- ระบบตัดจบอัตโนมัติเมื่อเงินหรือเวลาหมด (`autoEndExpiredSessions()`)
- พนักงานกดปุ่ม Force End หน้าร้าน (`StaffController.php`)

ทั้ง 3 ทางต้องใช้สูตรคำนวณเวลา ปรับสถานะเครื่อง ปรับเวลาแพ็กเกจ และบันทึกประวัติการเงินชุดเดียวกัน การรวมไว้ใน `app/Services/EndSeatSession.php` ช่วยป้องกันข้อผิดพลาดจากสูตรคิดเงินไม่ตรงกัน

SECURITY **คำถามที่ 2: ระบบป้องกันไม่ให้ผู้ใช้ทั่วไปแอบเข้าหน้าของพนักงานหรือแอดมินได้อย่างไร?**

คำตอบ: ใช้การป้องกัน 2 ระดับ:

- ระดับเส้นทาง (Route): ใน `routes/web.php` เส้นทางทั้งหมดถูกครอบด้วย `Route::middleware(['auth'])` เพื่อบังคับให้ผู้ใช้ต้องล็อกอินก่อนเสมอ
- ระดับตัวควบคุม (Controller): ใน `StaffController.php` มีเมธอด `authorizeStaff()` ตรวจสอบ `role === 'staff' || role === 'admin'` และใน `AdminController.php` มี `authorizeAdmin()` ตรวจสอบ `role === 'admin'` หากไม่ผ่านเงื่อนไข ระบบจะสั่ง `abort(403)` ปฏิเสธการเข้าถึงทันที

SECURITY **คำถามที่ 3: ระบบป้องกัน SQL Injection และ XSS (Cross-Site Scripting) อย่างไร?**

คำตอบ:

- **SQL Injection:** โค้ดทั้งหมดใช้ Eloquent ORM และ Query Builder ซึ่งส่งผ่านคำสั่งแบบ Parameter Binding (PDO Prepared Statements) อัตโนมัติ ข้อมูลที่ผู้ใช้กรอกเข้ามาจะถูกมองเป็นค่า Value ไม่ถูกนำไปประมวลผลเป็นคำสั่ง SQL
- **XSS:** ใน Blade Templates แสดงผลข้อความด้วย `{{ $variable }}` ซึ่ง Laravel จะครอบด้วยฟังก์ชัน `htmlspecialchars()` เพื่อแปลงอักขระพิเศษ (เช่น <, >, ") เป็น HTML Entities เสมอ

MVC **คำถามที่ 4: จงอธิบายเส้นทางของข้อมูล (Lifecycle Flow) เมื่อลูกค้ากดเช็คอินเปิดเครื่อง**

คำตอบ: เส้นทางของข้อมูลวิ่งข้าม Layer ดังนี้:

```
หน้าเว็บ (seat-map.blade.php) ส่งฟอร์ม POST /check-in พร้อม seat_id, billing_mode
    → routes/web.php จับคู่เรียก SeatController@checkIn
        → ตรวจสอบ Request Validation
            → ตรวจสอบสถานะและ Atomic Update ตาราง seats ('occupied')
                → Eloquent สร้างแถวใหม่ในตาราง seat_sessions (status: 'active')
                    → redirect กลับไปที่ Route 'dashboard' พร้อม Session Flash Success
```

5. เจาะลึกแนวคำถามรายบุคคล (ตามการแบ่งงาน 5 คน)

1. ปาล์ม (Dashboard, Seat Map, Auth)

- **ถาม:** ทำไมหน้า Dashboard ต้องเรียก `autoEndExpiredSessions()` ทุกครั้งที่โหลดหน้า?
- **ตอบ:** เนื่องจากโปรเจกต์นี้ไม่ได้เปิด Worker หรือ Cron Job ทำงานเบื้องหลังตลอดเวลา การเช็คใน DashboardController ก่อนดึงข้อมูลทำให้เมื่อผู้ใช้เปิดหน้าเว็บ ระบบจะตรวจสอบเวลาที่ใช้ไปเทียบกับเวลาของแพ็กเกจหรือเงินคงเหลือ หากหมดจะทำการตัดจบและคืนเครื่องให้ว่างทันที
- **ถาม:** สูตรคำนวณเวลาที่ใช้ไปในปัจจุบันคิดอย่างไร?
- **ตอบ:** ใช้ `diffInSeconds(Carbon::now())` จากเวลา `start_time` แล้วนำมาหาร 60 ปัดเศษขึ้นด้วย `ceil()` และกำหนดค่าต่ำสุดที่ 1 นาที

2. ตอง (Food Order, Cart, Wallet & Packages)

- **ถาม:** เวลาลูกค้าซื้อแพ็กเกจชั่วโมง มีขั้นตอนอย่างไรในฐานข้อมูล?
- **ตอบ:** ตรวจสอบเงินในกระเป๋า เปิด `DB::transaction()` หักเงินจาก `users.balance` บันทึกประวัติลง `wallet_transactions` (ref_type: package_purchase) และสร้างเรคคอร์ดใน `user_packages` กำหนดจำนวนนาที และตั้งวันหมดอายุล่วงหน้า 30 วัน
- **ถาม:** หน้าสั่งอาหารรู้ได้อย่างไรว่าจะต้องส่งอาหารไปที่เครื่องคอมพิวเตอร์เครื่องไหน?
- **ตอบ:** ระบบค้นหาจาก Active Session ของผู้ใช้ `SeatSession::where('user_id', $user->id)->where('status', 'active')->first()` แล้วดึง `seat_id` มาผูกกับออเดอร์อัตโนมัติ ลูกค้าไม่ต้องกรอกเอง

3. ดิว (Staff Monitor, Kitchen Queue, Admin Report)

- **ถาม:** ถ้าในครัวกดยกเลิกออเดอร์ (Cancel) ระบบคืนสต็อกสินค้าอย่างไร?
- **ตอบ:** ใน `StaffController.php` เมื่อสถานะเปลี่ยนเป็น `cancelled` ระบบจะวนลูปอ่านสินค้าใน `order→items` แล้วนำจำนวนชิ้นที่สั่งบวกกลับเข้า `products.stock_quantity` ทันที
- **ถาม:** รายงานยอดขายดึง 5 อันดับเมนูขายดีด้วยคำสั่ง SQL อะไร?
- **ตอบ:** JOIN ตาราง `order_items` , `orders` , และ `products` กรองบิลที่ชำระเงินแล้ว ใช้ `SUM(quantity) as total_qty` , `GROUP BY products.id` และจัดเรียงด้วย `orderByDesc('total_qty')→limit(5)`

4. บอล (Database: Master Data & Sessions)

- **ถาม:** อธิบายความสัมพันธ์ระหว่างตาราง `zones` , `seats` และ `seat_sessions`
- **ตอบ:** `Zone` มีความสัมพันธ์แบบ One-to-Many กับ `Seat` (1 โซนมีหลายเครื่อง) และ `Seat` มีความสัมพันธ์แบบ One-to-Many กับ `SeatSession` (1 เครื่องมีประวัติการเข้าใช้งานได้หลายครั้ง)
- **ถาม:** ในตาราง `seat_sessions` ทำไม `user_package_id` ต้องเป็น Nullable?
- **ตอบ:** เพื่อรองรับการเล่น 2 รูปแบบ ถ้าเล่นด้วยแพ็กเกจจะเก็บ ID ของ `user_packages` เพื่อตัดเวลา แต่ถ้าเล่นแบบคิดเงินตามจริง (Pay as you go) จะเป็น NULL เพื่อให้คิดเงินตามอัตรา `rate_snapshot`

5. นะโม (Database: Transactions, Orders & Products)

- **ถาม:** ในตาราง `orders` และ `order_items` คอลัมน์ `total_amount` กับ `subtotal` ต่างกันอย่างไร?
- **ตอบ:** `subtotal` อยู่ใน `order_items` เก็บราคารวมของสินค้ารายการนั้น (quantity * unit_price) ส่วน `total_amount` อยู่ใน `orders` เก็บผลรวมของทุกรายการในบิล
- **ถาม:** ทำไมตาราง `wallet_transactions` ต้องมีทั้ง `ref_type` และ `ref_id` ?
- **ตอบ:** เพื่อรองรับความสัมพันธ์แบบ Polymorphic โดยย่อ `ref_type` ระบุประเภทตารางต้นทาง (เช่น session, food_order, package_purchase) และ `ref_id` เก็บ Primary Key ของตารางนั้นๆ

6. แนวทางการเตรียมตัวและการสาธิตหน้าห้องสอบ (Demo Strategy)

การเตรียมพร้อมก่อนขึ้นพรีเซนต์

- **เปิด Terminal รันระบบไว้ล่วงหน้า:** รันคำสั่ง `php artisan serve`
- **เตรียมเบราว์เซอร์แยก 3 หน้าต่าง (สำหรับ 3 Role):**
  - หน้าต่าง 1 (ลูกค้า): บัญชี `customer1` (รหัสผ่าน `1`)
  - หน้าต่าง 2 (พนักงาน): บัญชี `staff` (รหัสผ่าน `password`)
  - หน้าต่าง 3 (ผู้ดูแลระบบ): บัญชี `admin` (รหัสผ่าน `password`)
- **เปิดโปรแกรมดูฐานข้อมูลจริง:** เปิดไฟล์ `database/database.sqlite` ด้วย SQLite Viewer หรือ DB Browser for SQLite เพื่อให้อาจารย์เห็นว่าข้อมูลในตารางถูกบันทึกจริง
- **คำสั่งไม้ตายเมื่ออาจารย์ถามเรื่องความถูกต้องของระบบ:** เปิด Terminal แล้วรันคำสั่ง `php artisan test` ซึ่งชุดทดสอบ Pest ทั้ง 53 เคสจะรันผ่านทั้งหมด ยืนยันว่าระบบไม่มีบั๊กและผ่านการทดสอบตามมาตรฐานสากล

ข้อควรระวังเมื่อ zip โปรเจกต์ส่งต่อหรือย้ายเครื่อง

หากย้ายไฟล์โปรเจกต์ไปเปิดที่เครื่องอื่นแล้วพบข้อผิดพลาด `filemtime(): stat failed` ใน `storage/framework/views` สาเหตุเกิดจากไฟล์แคชของ Blade และ Livewire จากเครื่องเดิมตกค้างอยู่ วิธีแก้คือเปิด Terminal ที่เครื่องใหม่แล้วรันคำสั่ง:

```
php artisan optimize:clear
```

ระบบจะล้างแคชเก่าทิ้งทั้งหมด และสร้างไฟล์แคชใหม่ที่ตรงกับเครื่องนั้นทันที